

Lesson 3 — Player In Control

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

Never move things in the handler

Wrong — stutters, ignores `dt`, no two keys at once:

```
document.addEventListener("keydown", e => {
  if (e.key === "ArrowRight") x += 10;
});
```

Right — the handler only records:

```
const keys = {};
document.addEventListener("keydown", e => {
  keys[e.key] = true;
  if (e.key.startsWith("Arrow"))
    e.preventDefault();
});
document.addEventListener("keyup", e => {
  keys[e.key] = false;
});
```

Then, once per frame:

```
function update(dt) {
  if (keys["ArrowRight"]) x += SPEED * dt;
  if (keys["ArrowLeft"]) x -= SPEED * dt;
}
```

Why the wrong way stutters

The OS repeats a held key for **typing**: one press, a pause of about half a second, then a repeat rate. Your game loop runs 60 times a second regardless. Poll, do not react.

Intent, not keys

```
const wantsRight =
  keys["ArrowRight"] || keys["d"];
```

One place to change when you add WASD, a gamepad, or touch.

Clamping

```
if (x < 0) x = 0;
if (x + w > canvas.width)
  x = canvas.width - w;
```

Mouse

```
const b = canvas.getBoundingClientRect();
mouseX = event.clientX - b.left;
```

`clientX` is relative to the **window**, so you must subtract the canvas position.

Words

Polling — asking every frame. Games do this.

Event-driven — reacting instantly. Web pages do this. `event.key` — the character: `"a"`, `"ArrowLeft"`. `event.code` — the physical key: `"KeyA"`.

Section A — Recall

A1. Why does moving the paddle inside a `keydown` handler make it stutter?

[2 marks]

A2. What is the difference between *polling* and *event-driven* input? Which do games use?

[2 marks]

A3. What does the `keys` object actually store? Give an example of what it might look like mid-game.

[2 marks]

A4. Why is `event.preventDefault()` needed for the arrow keys?

[2 marks]

A5. Explain the difference between an input and an *intent*, and give one practical reason to keep them apart.

[3 marks]

Section B — Predict the output

B1. The player presses the right arrow and holds it for exactly two seconds. `PADDLE_SPEED` is 420. How far does the paddle travel? Does your answer depend on the computer's frame rate?

[3 marks]

B2. What happens when the player presses the right arrow, holds it, and releases it? Describe the paddle's behaviour precisely.

[4 marks]

```
document.addEventListener("keydown", e => { keys[e.key] = true; });  
// no keyup listener at all
```

B3. The player holds **both** arrow keys at once. What does the paddle do, and why?

[3 marks]

```
if (keys["ArrowRight"]) { paddle.x += SPEED * dt; }  
if (keys["ArrowLeft"]) { paddle.x -= SPEED * dt; }
```

B4. A player moves right at 300 px/s and down at 300 px/s at the same time. How fast are they actually travelling? Show your working, and say why this is a problem for the game.

Section C — Find and fix the bug

C1. The paddle never moves at all, and the console shows no errors. What is wrong?

[3 marks]

```
document.addEventListener("keydown", e => { keys[e.key] = true; });
document.addEventListener("keyup", e => { keys[e.key] = false; });

function update(dt) {
  if (keys["arrowright"]) { paddle.x += SPEED * dt; }
}
```

C2. The paddle moves correctly but can be driven completely off the right-hand side of the screen, where it disappears. The author thought they had handled it. What is wrong, and what should the second line be?

[4 marks]

```
if (paddle.x < 0) { paddle.x = 0; }
if (paddle.x > canvas.width) { paddle.x = canvas.width; }
```

C3. This mouse-controlled paddle is always offset to the right of the pointer by exactly the same amount, and the offset changes if the page is scrolled. Name both bugs.

[4 marks]

```
canvas.addEventListener("mousemove", function (event) {
  paddle.x = event.clientX;
});
```

Section D — Build it

Start from your lesson 2 bouncing ball, or from `code/02-the-keys-object.html`.

- ☐ **Checkpoint 1.** Add the `keys` object with both `keydown` and `keyup` listeners. Draw the list of currently-held keys on the canvas so you can see it working. Do not move anything yet.
- ☐ **Checkpoint 2.** Draw a paddle near the bottom of the canvas, with its numbers in a `paddle` object.

- ☐ **Checkpoint 3.** Make the arrow keys move it, **inside** `update`, multiplied by `dt`. Hold the key down — it should glide, with no pause and no stutter.
- ☐ **Checkpoint 4.** Clamp it so it cannot leave the screen. Test by holding a key for several seconds.
- ☐ **Checkpoint 5.** Add WASD as a second control scheme, using the *intent* pattern — so `update` never mentions a key name directly.
- ☐ **Checkpoint 6.** Bring back your bouncing ball from lesson 2, so a ball bounces around while you steer the paddle. They will pass straight through each other; that is lesson 4's job.

IF THE PADDLE GETS STUCK MOVING

Your `keyup` listener is missing or misspelled, so the key never gets set back to `false`. Add a temporary line that draws the held-key list on screen and you will see it immediately.

Section E — Design challenge

NO RIGHT ANSWER

Marked on your reasoning, and on whether you name what your idea costs.

Your paddle starts at full speed the instant a key goes down and stops dead the instant it comes up. That is maximally *responsive*. It is not obviously *good*.

E1. Think of a platform game character. When you let go of right, do they stop instantly? Why not? What would it feel like if they did?

E2. In a rhythm game or a fighting game, instant response is essential and any smoothing would be a defect. What makes those different?

E3. Design it. Sketch how you would make the paddle speed up over about a quarter of a second, and slide a little after release. What extra numbers would the paddle have to remember?

E4. The hard one. A player presses jump a few milliseconds *before* they land. Nothing happens, the jump is lost, and it feels like the game ignored them. Almost every good platform game solves this. How would you? What would you store, and for how long?

Stretch goals

1. Two players on separate keys.
2. Support mouse and keyboard together — and decide what happens if both are used at once.
3. Acceleration and friction, tuned until it feels good.
4. **Just-pressed:** teleport the paddle to the centre when space is pressed, but only once per press, not continuously while held.
5. Draw every held key on screen, then find out how many your keyboard can report at once.